



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

M.C. René Adrián Dávila Pérez

Asignatura:

Programación orientada a objetos

Grupo:

07

No de práctica(s):

06

Integrante(s):

424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada:

1

Semestre:

2027-1

Fecha de entrega:

09 de septiembre de 2026

Observaciones:

Calificación:

Índice

1. Introducción	2
1.1. Planteamiento del Problema	2
1.2. Motivación	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Marco Teórico	2
3.1. Organización de Clases mediante Paquetes	2
3.2. Identificador Completamente Calificado e Importaciones	3
3.3. Estructura de Directorios de un Proyecto Java	3
3.4. Comandos y Banderas de Java	3
4. Desarrollo	3
4.1. Consulta Técnica sobre JAR y JavaDoc	3
4.2. Estructuración del Paquete General de Prácticas	3
4.3. Apuntes de comandos Java (Uso práctico)	4
4.4. Diagramas de Clase (UML agregados)	5
5. Resultados y Conclusiones	6

1. Introducción

1.1. Planteamiento del Problema

Cuando desarrollamos programas en Java y acumulamos múltiples archivos de código, tenerlos sueltos en una sola carpeta genera desorden, colisiones de nombres y complicaciones para mantener el proyecto. Para solucionar esto, necesitamos estructurar los componentes de forma jerárquica. En esta práctica abordamos dos aspectos fundamentales: por un lado, investigamos las buenas prácticas para empaquetar código en archivos ejecutables JAR y generar documentación automática con JavaDoc; por otro lado, configuramos el espacio de nombres oficial de la materia y separamos físicamente nuestros archivos fuente y binarios, organizando las cinco prácticas previas y el avance del proyecto hospitalario bajo las indicaciones del profesor.

1.2. Motivación

Trabajar con una estructura ordenada de paquetes y directorios separados nos permite mantener la limpieza del proyecto, evitando que los archivos fuente (.java) se mezclen con los binarios compilados (.class). Comprender cómo el compilador gestiona las rutas de origen y de destino nos ayuda a construir aplicaciones más limpias, formales y preparadas para trabajar en equipo.

2. Objetivos

2.1. Objetivo General

Organizar adecuadamente las clases según su funcionalidad bajo un paquete estructurado.

2.2. Objetivos Específicos

- Definir espacios de nombres únicos empleando la nomenclatura de la facultad (`mx.unam.fi.die.poo.g7`) y añadiendo subpaquetes específicos (`p1` a `p5` y `e1`).
- Implementar la separación física de directorios entre el código fuente (`src/`) y las clases compiladas (`test/`).
- Evaluar mediante consultas los estándares de empaquetamiento y documentación en Java.
- Compilar los archivos desde la terminal dirigiendo automáticamente los binarios hacia la estructura anidada de pruebas.

3. Marco Teórico

3.1. Organización de Clases mediante Paquetes

Un paquete en Java funciona como un contenedor que agrupa clases e interfaces afines bajo un mismo espacio de nombres. Sus objetivos consisten en agrupar clases relacionadas, evitar conflictos de nombres y regular el control de acceso [1, 3].

3.2. Identificador Completamente Calificado e Importaciones

El nombre completo o *Fully Qualified Name* representa la ruta exacta de una clase dentro del entorno de ejecución, combinando su paquete y su identificador [2]. Para utilizar clases de otros paquetes, empleamos la sentencia `import`.

3.3. Estructura de Directorios de un Proyecto Java

En el desarrollo formal se acostumbra dividir un proyecto en carpetas:

- **src:** Carpeta destinada a los archivos fuente `.java`.
- **test:** Directorio donde almacenamos los binarios `.class`.
- **lib:** Espacio para bibliotecas, dependencias o ejecutables `.jar`.
- **doc:** Carpeta reservada para la documentación técnica.

3.4. Comandos y Banderas de Java

Para operar esta arquitectura desde la terminal sin IDEs, requerimos el uso de los comandos base del JDK:

- **javac:** El compilador. Transforma el código humano a *bytecode*. Usamos la bandera `-d` (directory) para indicarle en qué carpeta raíz depositar los binarios respetando la jerarquía de paquetes.
- **java:** La máquina virtual. Ejecuta los binarios. Usa la bandera `-cp` (classpath) para indicarle en qué directorio debe empezar a buscar los paquetes.
- **javadoc:** Generador de documentación. Usamos `-d` para la carpeta de destino y `-sourcepath` para indicarle dónde buscar el código fuente original.
- **jar:** Empaquetador. Las banderas `c` (create), `f` (file) y `e` (entry point) sirven para crear el archivo, nombrarlo y definir la clase que contiene el `main`.

4. Desarrollo

4.1. Consulta Técnica sobre JAR y JavaDoc

Cada integrante consultó a una Inteligencia Artificial diferente acerca de las prácticas para empaquetar en JAR y documentar con JavaDoc. Las respuestas coincidieron en que un JAR ejecutable requiere declarar la clase principal, y que JavaDoc se construye a partir de comentarios estructurados con etiquetas como `@param` y `@return`.

4.2. Estructuración del Paquete General de Prácticas

Configuramos la arquitectura del paquete `mx.unam.fi.die.poo.g7` ajustándolo a la realidad de nuestras entregas:

1. **Subpaquetes independientes:** A cada práctica se le asignó su propio subpaquete (ej. `mx.unam.fi.die.poo.g7.p1`), declarándolo en la primera línea de cada archivo `.java`.
2. **Compilación dirigida:** Utilizamos el compilador enviando los archivos `.class` a la carpeta `test/`. El compilador generó automáticamente las subcarpetas necesarias (`test/mx/unam...`).
3. **Documentación aislada:** Ejecutamos JavaDoc dirigiendo la salida web a carpetas individuales nombradas según la clase (ej. `doc/Calculadora/`) para no sobrescribir los manuales PDF.
4. **Empaquetado selectivo:** Exclusivamente para el avance del proyecto hospitalario (Proyecto/E1), generamos el archivo `sistema-hospital.jar` y lo ubicamos en su carpeta `lib/`.

4.3. Apuntes de comandos Java (Uso práctico)

A continuación, detallamos la forma exacta en la que aplicamos los comandos para realizar las tareas descritas:

javac

Explicación: Lo utilizamos para compilar nuestros archivos de Java (`.java`) a archivos binarios (`.class`). Con la bandera `-d` le indicamos que guarde esos binarios en la carpeta `test` para que no se mezclen con el código fuente.

Terminal - Compilación

```
$ javac -d P1/test P1/src/mx/unam/fi/die/poo/g7/p1/*.java
```

java

Explicación: Sirve para ejecutar el programa que ya compilamos. Le pasamos `-cp` (class-path) para indicarle que nuestros archivos `.class` están en la carpeta `test`, y luego colocamos la ruta completa del paquete de nuestra clase principal.

Terminal - Ejecución

```
$ java -cp P1/test mx.unam.fi.die.poo.g7.p1.Calculadora
```

javadoc

Explicación: Lee los comentarios estructurados que dejamos en el código y construye páginas HTML con la documentación. La bandera `-d` hace que los guarde en una carpeta específica, evitando así desordenar nuestros manuales PDF.

Terminal - JavaDoc

```
$ javadoc -d P1/doc/Calculadora/ -sourcepath P1/src  
P1/src/mx/unam/fi/die/poo/g7/p1/Calculadora.java
```

jar

Explicación: Nos sirve para empaquetar todos nuestros archivos `.class` compilados en un solo archivo `.jar` (como si fuera un zip ejecutable). La `c` crea el archivo, la `f` le asigna el nombre y la `e` indica cuál es la clase principal que arranca el sistema.

Terminal - JAR

```
$ jar cfe Proyecto/E1/lib/sistema-hospital.jar  
mx.unam.fi.die.poo.g7.e1.Main -C Proyecto/E1/test mx
```

4.4. Diagramas de Clase (UML agregados)

Durante esta estructuración, formalizamos el diseño agregando los diagramas de clase correspondientes a los ejercicios previos:



Figura 1: Diagrama UML de la clase Hola.

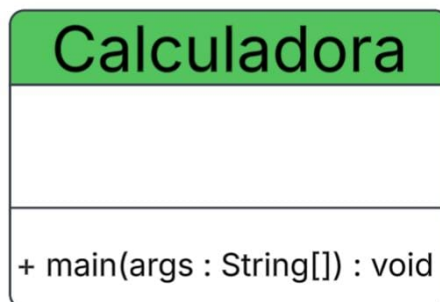


Figura 2: Diagrama UML de la clase Calculadora.

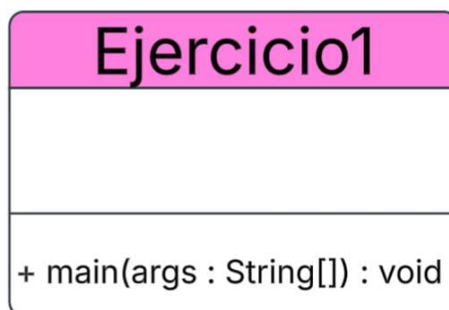


Figura 3: Diagrama UML del Ejercicio 1.

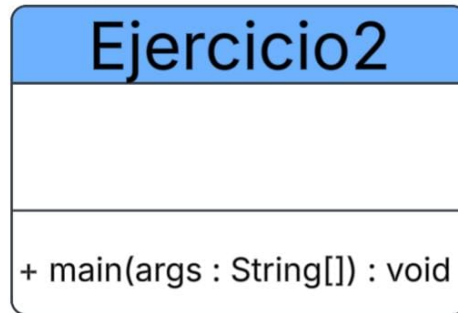


Figura 4: Diagrama UML del Ejercicio 2.

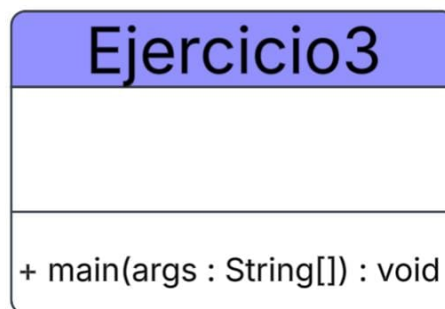


Figura 5: Diagrama UML del Ejercicio 3.

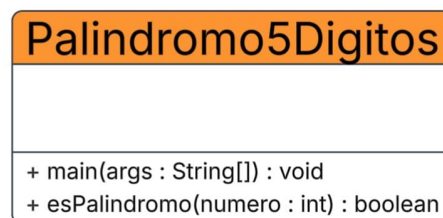


Figura 6: Diagrama UML de la clase Palindromo5Digitos.

5. Resultados y Conclusiones

El desarrollo de esta práctica nos permitió comprobar la utilidad de estructurar las clases bajo espacios de nombres definidos. Al establecer paquetes y subpaquetes, evitamos conflictos de nombres y logramos clasificar los programas de forma lógica.

Asimismo, al separar físicamente el código fuente en la carpeta de origen y los binarios compilados en la carpeta de pruebas, mantuvimos un entorno de trabajo limpio. Comprendimos que aislar la generación de JavaDoc por clases y encapsular los proyectos en archivos JAR facilita enormemente la distribución y mantenimiento del software.

Referencias

- [1] Oracle. *Creating and Using Packages*. The Java Tutorials. Disponible en: <https://docs.oracle.com/javase/tutorial/java/package/packages.html>
- [2] Real Python. *Fully Qualified Name (FQN)*. Computer Science Glossary. Disponible en: <https://realpython.com/ref/computer-science-glossary/fully-qualified-name-fqn/>
- [3] TayJava. *Packages and Access Modifiers in Java*. TayJava Technologies. Disponible en: <https://tayjava.com/technologies/packages-and-access-modifiers-in-java>
- [4] IBM. *Java JAR class files*. IBM Documentation. Disponible en: <https://www.ibm.com/docs/es/i/7.5.0?topic=platform-java-jar-class-files>